

API data dashboard

Module 6 - Lesson 2

Overview

Dashboards gather data from several sources and present it usefully. Fetching, cleaning, and aggregating data from APIs is the core of many products.

Key Concepts

- Define the question first: what decision should the dashboard inform?
- Fetch from multiple endpoints and normalize the responses into one shape.
- Clean data: drop nulls, fix types, and remove outliers before aggregating.
- Aggregate with grouping and counting to reveal patterns.
- Cache or persist results so repeated views don't hammer the API.
- Present results with charts and clear labels.

Example

```
import requests

def fetch_repos(org):
    res = requests.get(f"https://api.github.com/orgs/{org}/repos", timeout=10)
    res.raise_for_status()
    return [{"name": r["name"], "stars": r["stargazers_count"]} for r in res.json()]

def top_repos(org, limit=5):
    repos = fetch_repos(org)
    return sorted(repos, key=lambda r: r["stars"], reverse=True)[:limit]
```

Summary

Dashboards convert raw API responses into decisions. Normalize, clean, aggregate, and present - with caching so the data source isn't overused.

Practice Checklist

- Fetch two endpoints and merge their data into a single structure.
- Clean a messy dataset by dropping nulls and fixing types.
- Compute and print a top-N ranking from the cleaned data.